Problem Statement

An e-commerce platform introduces a redesigned checkout page intended to reduce friction and increase completed purchases. The goal is to estimate whether the change improves conversion rate relative to the current version.

Hypothesis:

Null hypothesis

```
The new checkout design does not change conversion rate
```

Alternative hypotheise

```
The new checkout design increases conversion rate
```

Reject null hypothesis if p-value < 0.05

Metric Selection

Primary Metric (OEC)

```
Conversion rate = Number of purchases/Number of users exposed
```

Secondary metrics

```
Add-to-cart rate
Checkout-start rate
Revenue per user
```

Guardrail metrics

```
Page load latency
Bounce rate
Session duration
```

1.Simulated Dataset Construction

```
Users               20000
Allocation          50/50
Baseline conversion 10%
Expected lift       +2 percentage points

So, p_C=0.10, p_T=0.12
```

```
import numpy as np
import pandas as pd

np.random.seed(42)

# experiment size
n_users = 20000

# assign variants
variant = np.random.choice(['control', 'treatment'], size=n_users)


# baseline probabilities
p_control = 0.10
p_treatment = 0.12

# simulate conversions
conversion = [
    np.random.binomial(1, p_control if v=='control' else p_treatment)
    for v in variant
]


# simulate guardrail metric: latency (ms)
latency = np.random.normal(250, 40, n_users)

df = pd.DataFrame({
    'user_id': range(n_users),
    'variant': variant,
    'conversion': conversion,
    'latency_ms': latency
})

df.head()
```

	user_id	variant	conversion	latency_ms
0	0	control	0	224.772307
1	1	treatment	0	335.612351
2	2	control	0	316.627163

3	3	control	0	279.237417
4	4	control	0	236.276963

Next steps: Generate code with df | New interactive sheet

2.Sanity checks before analysis

```
1. Check allocation rato
2. Conversion summary. Treatment should be slightly higher due to the simulat
3. Missing values
```

```
df['variant'].value_counts(normalize=True)
```

variant	proportion
control	0.501
treatment	0.499

dtype: float64

```
df.groupby('variant')['conversion'].mean()
```

variant	conversion
control	0.104092
treatment	0.118838

dtype: float64

```
df.isnull().sum()
```

	0
user_id	0
variant	0
conversion	0
latency_ms	0

dtype: int64

3.Statistical Testing

```
Estimate whether the observed difference in conversion rates reflects a real
```

3.1 Estimate conversion rates

```
control_rate = df[df['variant']=='control']['conversion'].mean()
treatment_rate = df[df['variant']=='treatment']['conversion'].mean()

control_rate, treatment_rate
```

```
(np.float64(0.10409181636726547), np.float64(0.1188376753507014))
```

```
lift = treatment_rate - control_rate
lift
```

```
np.float64(0.01474585898343593)
```

```
relative_lift = lift / control_rate
relative_lift
```

```
np.float64(0.1416620393231333)
```

3.2 Two-proportion Z-test

```
from statsmodels.stats.proportion import proportions_ztest

control = df[df['variant']=='control']['conversion']
treatment = df[df['variant']=='treatment']['conversion']

successes = [treatment.sum(), control.sum()]
observations = [len(treatment), len(control)]

z_stat, p_value = proportions_ztest(successes, observations, alternative=

z_stat, p_value
```

```
(np.float64(3.3133951936963686), np.float64(0.0004608532194930384))
```

3.3 Interpret p-value

```
if p_value < 0.05:
    print('Reject null hypothesis.\nStatistically significant improvement
else:
    print('Fail to reject null hypothesis')
```

```
    print("Fail to reject null hypothesis")
```

```
Reject null hypothesis.
Statistically significant improvement in conversion rate.
```

3.4 Confidence interval for treatment effect

```
from statsmodels.stats.proportion import confint_proportions_2indep

ci_low, ci_high = confint_proportions_2indep(
    treatment.sum(),
    len(treatment),
    control.sum(),
    len(control),
    method='wald'
)

ci_low, ci_high
```

```
(np.float64(0.006024664489028032), np.float64(0.02346705347784383))
```

From above we infer true lift likely between 0.6% and 2.35% with 95% confidence.

Since the interval does not include 0, evidence supports treatment effectiveness.

3.5 Validate Guardrail Metric (Latency)

```
from scipy.stats import ttest_ind

lat_control = df[df['variant']=='control']['latency_ms']
lat_treatment = df[df['variant']=='treatment']['latency_ms']

t_stat, latency_p = ttest_ind(lat_treatment, lat_control)

lat_control.mean(), lat_treatment.mean(), latency_p
```

```
(np.float64(249.94324413094043),
 np.float64(249.79241942461005),
 np.float64(0.7899363908695796))
```

latency_p, the p-value, is larger than 0.05 so no evidence to indicate latency changed.

3.6 Sample Ration Mismatch(SRM) Check

```
We expected a 50/50 split. We test using chi-square test
```

```
from scipy.stats import chisquare

observed = df['variant'].value_counts().values
expected = [len(df)/2, len(df)/2]

chisquare(observed, expected)
```

```
Power_divergenceResult(statistic=np.float64(0.08),
pvalue=np.float64(0.7772974107895214))
```

p-value is larger than 0.05 so experiment is valid, there is no evidence to suggest ratio mismatch.

4. Power Analysis and Sample Size calculation

Power analysis determines whether an experiment has enough users to reliably detect a meaningful treatment effect. This section is essential in production experimentation workflows and is expected in strong A/B testing portfolios.

We answer two questions:

```
Did our experiment have sufficient statistical power?
How many users should we plan for future experiments?
```

4.1 Define Key parameters

α = 0.05

power = 0.80

baseline conversion(control rate) = 0.10

MDE(minimum detectable effect) = +0.02

$MDE = p_T - p_C$ Interpretation: Smallest improvement worth detecting operationally

4.2 Compute Required Sample Size

Use a two-proportion power test

```
from statsmodels.stats.power import NormalIndPower
from statsmodels.stats.proportion import proportion_effectsize

baseline_rate = 0.10
treatment_rate = 0.12
```

```
effect_size = proportion_effectsize(baseline_rate, treatment_rate)

analysis = NormalIndPower()

sample_size = analysis.solve_power(
    effect_size=effect_size,
    power=0.80,
    alpha=0.05,
    ratio=1
)

sample_size
```

```
3834.595739884031
```

So we should have twice as many users ≈ 7668 users. Dataset has 20,000 users so power is sufficient.

4.3 Estimate achieved power of the experiment

```
actual_n = len(df[df['variant']=='control'])

achieved_power = analysis.power(
    effect_size=effect_size,
    nobs1=actual_n,
    alpha=0.05,
    ratio=1
)

achieved_power
```

```
np.float64(0.9948970778185491)
```

Interpretation:

Experiment highly sensitive to the effect size tested because power is greater than 0.95.

4.4 Sensitivity Analysis across effect sizes

```
import numpy as np

mde_range = np.linspace(0.005, 0.03, 20)

powers = []

for mde in mde_range:
    es = proportion_effectsize(0.10, 0.10 + mde)
```

```
    power_val = analysis.power(
        effect_size=es,
        nobs1=actual_n,
        alpha=0.05,
        ratio=1
    )
    powers.append(power_val)

list(zip(mde_range, powers))[:10]
```

```
[(np.float64(0.005), np.float64(0.21475858380408788)),
 (np.float64(0.00631578947368421), np.float64(0.3123428510654556)),
 (np.float64(0.0076315789473684215), np.float64(0.4252405955895155)),
 (np.float64(0.008947368421052631), np.float64(0.5442049129136516)),
 (np.float64(0.010263157894736842), np.float64(0.6587703520934408)),
 (np.float64(0.011578947368421053), np.float64(0.7598023566903221)),
 (np.float64(0.012894736842105264), np.float64(0.8415163378952372)),
 (np.float64(0.014210526315789472), np.float64(0.9022118428151359)),
 (np.float64(0.015526315789473683), np.float64(0.9436686967092467)),
 (np.float64(0.016842105263157894), np.float64(0.9697393184553874))]
```

We see that at 0.0155 we acheive power of ≈ 0.95. So for lift of ≈ 0.0155 and above sensitivity is high.

5. Heterogenous Treatment Effects(Segmented Analysis/ CATE)

5.1 Why Segmented Analysis?

Segment	Effect
New users	+4% lift
Returning users	0% lift

So overall lift could be 2% but that does not capture where value is generated.

5.2 Add Simulated Segmentation Variables

We extend dataset with realistic experiment covariates

- device type
- user tenure
- traffice source

```
import numpy as np

np.random.seed(42)

df['device'] = np.random.choice(
```

```
df['device'] = np.random.choice(
    ['mobile', 'desktop'],
    size=len(df),
    p=[0.6, 0.4]
)

df['user_type'] = np.random.choice(
    ['new', 'returning'],
    size=len(df),
    p=[0.5, 0.5]
)

df['traffic_source'] = np.random.choice(
    ['organic', 'ads', 'referral'],
    size=len(df),
    p=[0.5, 0.3, 0.2]
)

df.head()
```

	user_id	variant	conversion	latency_ms	device	user_type	traffic_so
0	0	control	0	224.772307	mobile	returning	or
1	1	treatment	0	335.612351	desktop	new	or
2	2	control	0	316.627163	desktop	new	or
3	3	control	0	279.237417	mobile	returning	or
4	4	control	0	236.276963	mobile	new	or

Next steps: Generate code with df New interactive sheet

5.3 Estimate Treatment Effect by Segment

```
df.groupby(['device', 'variant'])['conversion'].mean().unstack()
```

variant	control	treatment
device		
desktop	0.097851	0.117485
mobile	0.108162	0.119739

```
device_effect = (
    df.groupby(['device', 'variant'])['conversion']
    .mean()
    .unstack()
)

device_effect['lift'] = (
```

```
    device_effect['treatment'] - device_effect['control']
)

device_effect
```

variant device	control	treatment	lift
desktop	0.097851	0.117485	0.019634
mobile	0.108162	0.119739	0.011578

Next steps: Generate code with `device_effect` New interactive sheet

We see lift is almost double for desktop users.

5.4 Statistical Testing within Segments

Mobile Users

```
from statsmodels.stats.proportion import proportions_ztest

mobile = df[df['device']=='mobile']

successes = [
    mobile[mobile['variant']=='treatment']['conversion'].sum(),
    mobile[mobile['variant']=='control']['conversion'].sum()
]

observations = [
    len(mobile[mobile['variant']=='treatment']),
    len(mobile[mobile['variant']=='control'])
]

proportions_ztest(successes, observations)
```

```
(np.float64(2.0003805806938066), np.float64(0.04545918369467556))
```

Desktop users

```
from statsmodels.stats.proportion import proportions_ztest

desktop = df[df['device']=='desktop']

successes = [
    desktop[desktop['variant']=='treatment']['conversion'].sum(),
    desktop[desktop['variant']=='control']['conversion'].sum()
]
```

```
observations = [
    len(desktop[desktop['variant']=='treatment']),
    len(desktop[desktop['variant']=='control'])
]

proportions_ztest(successes, observations)
```

```
(np.float64(2.822874407357654), np.float64(0.0047595212362878695))
```

We see p-value is less than 0.05 for both mobile users and desktop users but p-value of desktop users is significantly smaller but for mobile users it is close to the significance level.

Effect seems localized to desktop users

5.5 User Tenure Treatment effect

```
tenure_effect = (
    df.groupby(['user_type', 'variant'])['conversion']
    .mean()
    .unstack()
)

tenure_effect['lift'] = (
    tenure_effect['treatment'] - tenure_effect['control']
)

tenure_effect
```

variant	control	treatment	lift
user_type			
new	0.102696	0.123227	0.02053
returning	0.105506	0.114546	0.00904

Next steps: Generate code with `tenure_effect` New interactive sheet

For new users the lift is more than double that of returning users.

One can interpret this as new interface helps onboarding users more.

5.6 Traffic Source Treatment Effect

```
traffic_effect = (
    df.groupby(['traffic_source', 'variant'])['conversion']
    .mean()
    .unstack()
```

```
    .unstack()
)

traffic_effect['lift'] = (
    traffic_effect['treatment'] - traffic_effect['control']
)

traffic_effect
```

variant	control	treatment	lift
traffic_source			
ads	0.105664	0.121527	0.015862
organic	0.104592	0.117635	0.013043
referral	0.100583	0.117829	0.017246

Next steps: Generate code with `traffic_effect` New interactive sheet

All traffic sources show positive lift (~1.3–1.7%), but statistical significance cannot be determined without segment-level hypothesis tests. Given smaller per-segment sample sizes, these effects may or may not be reliable.

The referral segment shows the largest observed lift making it most likely to reach statistical significance if a real heterogeneous treatment effect exists.

We do segment-level hypothesis test for this segment.

5.6.1

```
from statsmodels.stats.proportion import proportions_ztest

referral = df[df['traffic_source'] == "referral"]

successes = [
    referral[referral['variant'] == "treatment"]['conversion'].sum(),
    referral[referral['variant'] == "control"]['conversion'].sum()
]

observations = [
    len(referral[referral['variant'] == "treatment"]),
    len(referral[referral['variant'] == "control"])
]

z_stat, p_value = proportions_ztest(
    successes,
    observations,
    alternative='larger'
```

```
)

z_stat, p_value
```

```
(np.float64(1.7480862140767708), np.float64(0.040224549887594475))
```

5.6.2

p-value is smaller than 0.05 so we can conclude referral users experience a statistically significant improvement in conversion rate. This supports heterogeneous treatment effects by traffic source.

5.7 Estimating Conditional Average Treatment Effects(CATE)

$CATE(x) = E[Y(1) - Y(0)|X = x]$ Where

- $X =$ segment variable
- Y = outcome

```
traffic_effect
```

variant	control	treatment	lift
traffic_source			
ads	0.105664	0.121527	0.015862
organic	0.104592	0.117635	0.013043
referral	0.100583	0.117829	0.017246

Next steps: Generate code with `traffic_effect` New interactive sheet

6 Experiment Validity Check-Sample Ratio Mismatch(SRM)

```
from scipy.stats import chisquare

observed = df['variant'].value_counts().values
expected = [len(df)/2, len(df)/2]

chisquare(observed, expected)
```

```
Power_divergenceResult(statistic=np.float64(0.08),
pvalue=np.float64(0.7772974107895214))
```

p-value is much larger than 0.05 so no evidence of SRM